

TP MongoDB : FraudShield Banking - Système de Détection de Fraude

Informations générales

Niveau : Bac+3 **Durée estimée :** 4-6 heures **Prérequis :** Cours MongoDB Jour 1 & Jour 2 **Rendu :** Fichier contenant vos requêtes MongoDB et vos réponses

Contexte métier

Vous êtes data analyst chez **FraudShield**, une fintech spécialisée dans la détection de fraudes bancaires en temps réel. Votre mission consiste à analyser les transactions financières de plusieurs institutions bancaires partenaires afin d'identifier les patterns de fraude et d'optimiser le système de détection.

Le département Risk Management vous a fourni un dataset historique contenant des millions de transactions bancaires, dont certaines ont été identifiées comme frauduleuses. Votre objectif est d'explorer ces données, d'identifier les tendances, et de proposer des insights actionnables pour améliorer la détection.

Enjeux business :

- Réduire le taux de faux positifs (transactions légitimes bloquées)
 - Détecter les fraudes avant qu'elles n'impactent les clients
 - Comprendre les comportements typiques des fraudeurs
 - Optimiser les performances du système de requêtage en temps réel
-

Partie 1 : Installation et Import du Dataset

1.1 Préparation de l'environnement

Question 1.1.1 Créez une base de données MongoDB nommée `fraudshield_banking`. Documentez la commande utilisée.

Question 1.1.2 Le fichier CSV contient des données brutes. Avant d'importer, identifiez dans la documentation MongoDB la commande permettant d'importer un fichier CSV. Quels sont les paramètres essentiels à utiliser ?

 **Indice :** Cherchez "import CSV MongoDB" dans la documentation officielle

Question 1.1.3 Importez le dataset `FraudShield_Banking_Data.csv` dans une collection nommée `transactions`.

Répondez aux questions suivantes :

- Combien de documents ont été importés ?
- Quel est le type de données de chaque champ après l'import ?

- Pourquoi certains champs numériques pourraient-ils être importés comme des strings ? Comment corriger cela ?

 **Indice :** Consultez les options de `mongoimport`, notamment `--type` et `--headerline`

1.2 Validation de l'import

Question 1.2.1 Écrivez une requête qui affiche les 5 premières transactions de la collection. Analysez la structure des documents.

Question 1.2.2 Certains champs contiennent des valeurs "Yes"/"No". Proposez une stratégie pour convertir ces champs en booléens. Implémentez cette conversion pour au moins 3 champs pertinents.

 **Indice :** Regardez les opérateurs de mise à jour et la conversion de types dans MongoDB

Partie 2 : Exploration et CRUD - Comprendre les Données

2.1 Opérations de lecture basiques

Question 2.1.1 Trouvez le nombre total de transactions frauduleuses dans le dataset. Comparez-le au nombre total de transactions. Calculez le taux de fraude en pourcentage.

Question 2.1.2 Identifiez la transaction avec le montant le plus élevé. Est-elle frauduleuse ? Affichez tous ses détails.

Question 2.1.3 Listez les 10 clients (Customer_ID) ayant effectué le plus grand nombre de transactions. Affichez uniquement leur ID et le nombre de leurs transactions.

 **Astuce :** Vous aurez besoin d'utiliser une méthode d'agrégation pour cette question

2.2 Filtrage avancé

Question 2.2.1 Trouvez toutes les transactions qui remplissent SIMULTANÉMENT ces critères :

- Montant supérieur à 5 millions
- Transaction internationale
- Effectuée avec une carte de crédit
- Compte ayant un historique de fraude (Previous_Fraud_Count > 0)

Combien de transactions correspondent ? Quel est le taux de fraude parmi celles-ci ?

Question 2.2.2 Identifiez les transactions effectuées à une heure inhabituelle (Unusual_Time_Transaction) ET à plus de 100 km du domicile du client (Distance_From_Home). Parmi ces transactions, combien sont frauduleuses ?

Question 2.2.3 Recherchez les transactions effectuées dans les catégories de marchands suivantes : "Electronics", "Jewelry", "Luxury Goods". Affichez uniquement le Transaction_ID, le montant, la catégorie et le statut de fraude.

 **Indice :** Utilisez l'opérateur approprié pour rechercher dans une liste de valeurs

2.3 Opérations de mise à jour

Question 2.3.1 Le système a détecté une erreur : toutes les transactions du client "CUST0012345" datant du 15/01/2024 ont été incorrectement marquées comme frauduleuses. Corrigez cette erreur en les marquant comme légitimes.

Question 2.3.2 Ajoutez un nouveau champ `risk_level` à toutes les transactions. Ce champ doit être calculé selon ces règles :

- "HIGH" si : (montant > 10 millions) OU (Previous_Fraud_Count > 2) OU (Distance_From_Home > 500)
- "MEDIUM" si : (montant > 5 millions) OU (Transaction internationale) OU (Failed_Transaction_Count > 3)
- "LOW" pour tous les autres cas

Implémentez cette logique en plusieurs étapes de mise à jour.

Question 2.3.3 Pour des raisons de conformité RGPD, vous devez anonymiser les IP_Address de toutes les transactions de plus de 2 ans. Remplacez-les par "ANONYMIZED".

 **Réflexion :** Comment identifiez-vous les transactions de plus de 2 ans ? Quelle opérateur de date utilisez-vous ?

2.4 Opérations de suppression

Question 2.4.1 Créez une collection `archive_transactions` et déplacez-y toutes les transactions frauduleuses ayant plus de 3 tentatives échouées (Failed_Transaction_Count >= 3). Après vérification, supprimez-les de la collection principale.

 **Attention :** Ne supprimez rien sans avoir fait une copie !



Partie 3 : Requêtes Avancées - Patterns de Fraude

3.1 Analyse des patterns temporels

Question 3.1.1 Identifiez les heures de la journée (0-23h) où le nombre de fraudes est le plus élevé. Présentez les résultats triés par nombre de fraudes décroissant.

 **Indice :** Vous devrez extraire l'heure du champ Transaction_Time et regrouper par cette valeur

Question 3.1.2 Trouvez les clients qui ont effectué plus de 10 transactions en une seule journée ET dont au moins une transaction a été frauduleuse. Affichez leur Customer_ID et le nombre total de transactions.

3.2 Analyse géographique

Question 3.2.1 Identifiez les 5 localisations de transactions (Transaction_Location) avec le taux de fraude le plus élevé. Pour chaque localisation, affichez :

- Le nom de la localisation
- Le nombre total de transactions
- Le nombre de fraudes

- Le taux de fraude en pourcentage

Question 3.2.2 Trouvez toutes les transactions où la localisation de transaction est différente de la localisation du domicile du client (Customer_Home_Location), avec une distance supérieure à 200 km. Calculez le taux de fraude pour ces transactions "à distance".

3.3 Analyse des marchands

Question 3.3.1 Identifiez les 10 marchands (Merchant_ID) avec le montant total de transactions frauduleuses le plus élevé. Pour chaque marchand, calculez :

- Le montant total des fraudes
- Le nombre de fraudes
- Le montant moyen par fraude

Question 3.3.2 Trouvez les catégories de marchands (Merchant_Category) où les clients utilisent préférentiellement des cartes de crédit vs des cartes de débit. Présentez les résultats avec le ratio crédit/débit pour chaque catégorie.

3.4 Analyse comportementale

Question 3.4.1 Identifiez les clients dont le montant de transaction actuel dépasse de plus de 300% leur montant moyen de transaction (Avg_Transaction_Amount). Quel est le taux de fraude pour ces "transactions exceptionnelles" ?

Question 3.4.2 Trouvez les transactions où le client a utilisé un nouveau marchand (Is_New_Merchant) ET où c'est une transaction internationale. Analysez si ces facteurs combinés sont des indicateurs forts de fraude.

Question 3.4.3 Créez une requête qui identifie les "transactions suspectes" selon les critères suivants (au moins 3 doivent être vrais) :

- Montant > 2x la moyenne du client
- Transaction à heure inhabituelle
- Nouveau marchand
- Transaction internationale
- Distance du domicile > 100 km
- Plus de 5 transactions dans la journée

Combien de transactions correspondent ? Quel est leur taux de fraude ?

Partie 4 : Indexation et Performance

4.1 Analyse des performances sans index

Question 4.1.1 Exécutez la requête suivante et analysez ses performances avec

`.explain("executionStats") :`

```
db.transactions.find({
  Transaction_Amount: { $gt: 5 },
  Fraud_Label: "Fraud",
  Is_International_Transaction: "Yes"
})
```

Notez les métriques suivantes :

- executionTimeMillis
- totalDocsExamined
- nReturned
- Le stage utilisé (COLLSCAN ou IXSCAN)

Question 4.1.2 Sans créer d'index, proposez 3 requêtes fréquentes dans un système de détection de fraude en temps réel. Analysez leurs performances actuelles.

4.2 Stratégie d'indexation

Question 4.2.1 Créez un index sur le champ **Fraud_Label**. Ré-exécutez la requête de la question 4.1.1 et comparez les performances. Quelle amélioration observez-vous ?

Question 4.2.2 En vous basant sur la règle ESR (Equality, Sort, Range), créez un index composé optimal pour cette requête :

```
db.transactions.find({
  Customer_ID: "CUST0012345",
  Transaction_Amount: { $gte: 1, $lte: 10 }
}).sort({ Transaction_Date: -1 })
```

Justifiez l'ordre des champs dans votre index.

 **Rappel :** ESR = Equality (=) puis Sort puis Range (<, >, etc.)

Question 4.2.3 Créez un index qui optimise les recherches par localisation ET catégorie de marchand. Testez son efficacité sur une requête pertinente.

Question 4.2.4 Le champ **IP_Address** doit être unique pour des raisons de sécurité. Créez l'index approprié. Que se passe-t-il s'il existe des doublons ? Comment gérer cette situation ?

4.3 Index avancés

Question 4.3.1 Créez un index partiel qui indexe uniquement les transactions frauduleuses avec un montant supérieur à 1 million. Expliquez pourquoi ce type d'index est utile.

 **Indice :** Cherchez "partial index" dans la documentation MongoDB

Question 4.3.2 Certains clients n'ont pas de Previous_Fraud_Count (champ manquant). Créez un index sparse approprié pour ce champ. Quelle est la différence avec un index normal ?

Question 4.3.3 Listez tous les index de votre collection avec leurs tailles. Identifiez s'il existe des index inutilisés ou redondants. Proposez un nettoyage.

 **Indice** : Regardez la commande `db.collection.getIndexes()` et `$indexStats`

4.4 Index couvrants

Question 4.4.1 Créez un index qui permet d'exécuter cette requête en tant que "covered query" (requête couverte) :

```
db.transactions.find(
  { Customer_ID: "CUST0012345" },
  { Customer_ID: 1, Transaction_Amount: 1, Transaction_Date: 1, _id: 0 }
)
```

Vérifiez avec `.explain()` que `totalDocsExamined` est à 0.



Partie 5 : Agrégation et Analyse Avancée

5.1 Pipelines d'agrégation basiques

Question 5.1.1 Créez un pipeline d'agrégation qui calcule, pour chaque type de carte (Card_Type), le montant total des transactions, le montant moyen, et le nombre de transactions. Triez par montant total décroissant.

Question 5.1.2 Calculez pour chaque catégorie de marchand :

- Le nombre total de transactions
- Le nombre de fraudes
- Le taux de fraude (en %)
- Le montant moyen des fraudes

Affichez uniquement les catégories avec un taux de fraude supérieur à 10%.

Question 5.1.3 Identifiez les 20 clients ayant le solde de compte le plus élevé (Account_Balance). Pour chacun, affichez leur Customer_ID, leur solde, et le nombre total de leurs transactions.

5.2 Groupements et calculs complexes

Question 5.2.1 Créez une analyse hebdomadaire des fraudes : pour chaque semaine, calculez :

- Le nombre total de transactions
- Le nombre de fraudes
- Le montant total des fraudes
- Le montant moyen par transaction frauduleuse

 **Indice** : Utilisez les opérateurs de date pour extraire la semaine

Question 5.2.2 Analysez le comportement des clients selon leur historique de fraude :

- Groupe 1 : Previous_Fraud_Count = 0 (clients propres)
- Groupe 2 : Previous_Fraud_Count = 1-2 (risque modéré)
- Groupe 3 : Previous_Fraud_Count > 2 (haut risque)

Pour chaque groupe, calculez le taux de fraude actuel et le montant moyen des transactions.

Question 5.2.3 Créez un pipeline qui identifie les "heures de pointe de fraude". Pour chaque heure de la journée (0-23h), calculez le ratio (fraudes/transactions totales). Identifiez les 5 heures les plus risquées.

5.3 Lookups et jointures

Question 5.3.1 Créez une collection **merchants** contenant des informations fictives sur au moins 10 marchands (Merchant_ID, nom, adresse, catégorie, date d'ouverture). Puis créez un pipeline qui joint les transactions avec les informations des marchands et affiche un rapport complet pour les transactions frauduleuses.

 **Indice :** Utilisez l'opérateur **\$lookup**

Question 5.3.2 Créez une collection **customers** avec des informations fictives sur les clients. Créez ensuite un pipeline qui génère un "profil de risque client" comprenant :

- Informations client de base
- Nombre total de transactions
- Nombre de fraudes historiques
- Montant moyen de transaction
- Score de risque calculé

5.4 Analyses prédictives

Question 5.4.1 Créez un pipeline qui identifie les transactions ayant un "score de suspicion" élevé basé sur la combinaison de facteurs suivants (attribuez des points) :

- Transaction internationale (+3 points)
- Nouveau marchand (+2 points)
- Heure inhabituelle (+2 points)
- Distance > 100km (+2 points)
- Montant > 2x moyenne client (+3 points)
- Failed_Transaction_Count > 0 (+1 point par échec)

Affichez les 50 transactions avec le score le plus élevé et vérifiez combien sont réellement frauduleuses.

Question 5.4.2 Créez une vue matérialisée (collection calculée avec **\$out** ou **\$merge**) nommée **daily_fraud_stats** qui agrège les statistiques quotidiennes :

- Date
- Nombre de transactions
- Nombre de fraudes
- Montant total des fraudes
- Taux de fraude
- Top 3 des catégories de marchands par nombre de fraudes

Cette vue doit être mise à jour quotidiennement.

Partie 6 : Requêtes Expertes et Optimisation

6.1 Requêtes complexes multi-critères

Question 6.1.1 Créez une requête qui identifie les "fraudes en série" : des clients ayant effectué au moins 3 transactions frauduleuses dans une fenêtre de 7 jours. Pour chaque client, affichez :

- Customer_ID
- Nombre de fraudes dans la période
- Montant total des fraudes
- Dates des fraudes

Question 6.1.2 Identifiez les "patterns de blanchiment" potentiels : séquences de transactions d'un même client avec :

- Montants croissants (chaque transaction > précédente)
- Toutes dans la même catégorie de marchand
- Au moins 4 transactions dans la séquence
- Montant total de la séquence > 20 millions

6.2 Performance ultime

Question 6.2.1 Optimisez au maximum cette requête fréquente du système de détection temps réel :

```
db.transactions.find({
  Customer_ID: "CUST0012345",
  Transaction_Date: { $gte: ISODate("2024-01-01"), $lte: ISODate("2024-12-31") },
  Fraud_Label: "Fraud"
}).sort({ Transaction_Amount: -1 }).limit(10)
```

Créez les index nécessaires et documentez l'amélioration de performance avec des chiffres précis (avant/après).

Question 6.2.2 Le système doit répondre en moins de 10ms à cette question : "Ce client a-t-il déjà commis une fraude ?". Créez une stratégie d'indexation et de requête optimale. Mesurez le temps de réponse.

6.3 Vue et sécurité

Question 6.3.1 Créez une vue `public_transactions` qui masque les informations sensibles (IP_Address, Device_ID, Customer_Home_Location) mais garde les informations nécessaires pour l'analyse de fraude. Cette vue ne doit montrer que les transactions des 30 derniers jours.

Question 6.3.2 Créez une vue `fraud_summary_by_merchant_category` qui affiche pour chaque catégorie de marchand un résumé agrégé (sans révéler les transactions individuelles).

Partie 7 : Rapport d'Analyse et Recommandations

7.1 Analyse finale

Rédigez un rapport (1-2 pages) comprenant :

1. **Executive Summary** : Les 5 insights les plus importants découverts dans les données
2. **Patterns de fraude identifiés** :
 - Quelles sont les caractéristiques communes des transactions frauduleuses ?
 - Existe-t-il des patterns temporels, géographiques ou comportementaux ?
3. **Recommandations techniques** :
 - Quels index recommandez-vous pour la production ?
 - Quelles requêtes d'agrégation devraient être pré-calculées ?
 - Comment optimiser les performances du système ?
4. **Recommandations business** :
 - Quelles règles de détection proposez-vous ?
 - Comment réduire les faux positifs ?
 - Quels indicateurs devraient alerter en temps réel ?

7.2 Dashboard temps réel

Proposez la structure d'un dashboard de monitoring pour l'équipe Risk Management. Pour chaque métrique, fournissez la requête MongoDB correspondante :

1. Nombre de transactions frauduleuses dans les dernières 24h
2. Top 5 des catégories de marchands à risque (cette semaine)
3. Liste des clients avec score de risque critique
4. Montant total des fraudes détectées (aujourd'hui vs hier)
5. Taux de fraude en temps réel (glissant sur 1h)



Critères d'Évaluation

Votre travail sera évalué selon les critères suivants :

Critère	Points	Description
Exactitude des requêtes	30%	Les requêtes fonctionnent et retournent les bons résultats
Optimisation	20%	Utilisation appropriée des index et bonnes pratiques de performance
Complexité	20%	Maîtrise des agrégations complexes et pipelines avancés
Documentation	15%	Code commenté, explications claires, justifications

Critère	Points	Description
Analyse critique	15%	Qualité du rapport, pertinence des insights et recommandations

Ressources Utiles

- [Documentation officielle MongoDB](#)
- [Aggregation Pipeline](#)
- [Index Strategies](#)
- [Query Optimization](#)
- [MongoDB University](#) - Cours gratuits

Consignes Importantes

1. **Travail individuel** : Ce TP est à réaliser seul
2. **Documentation** : Toutes vos requêtes doivent être commentées
3. **Tests** : Vérifiez vos résultats avec des requêtes de validation
4. **Performance** : Utilisez systématiquement `.explain()` pour vos optimisations
5. **Sauvegarde** : Créez des backups avant toute opération de suppression massive
6. **Code propre** : Formatez vos requêtes pour la lisibilité

17 Livrables

À rendre dans un dossier ZIP nommé `Nom_Prenom_TP_MongoDB.zip` contenant :

1. **queries.js** : Fichier JavaScript avec toutes vos requêtes numérotées et commentées
2. **rapport.pdf** : Votre rapport d'analyse (Partie 7)
3. **screenshots/** : Captures d'écran des résultats de vos analyses de performance (explain)
4. **README.md** : Instructions pour reproduire vos résultats

Bon courage ! 🚀

N'oubliez pas : en cas de doute, la documentation officielle est votre meilleure alliée.